PYTHON TKINTER GUI WORKSHOP

Beginner Practical Handbook

Building a Student Registration Desktop Application - Step by Step

Duration: 2 Hours	**Level: Beginner**	**Project: Student Registration GUI**

Purpose of this handbook

This handbook is designed so a student can follow the workshop from setup to a working Tkinter application without needing separate notes. Every operation is explained, then demonstrated with code, followed by a small practical task.

1. Workshop Learning Outcomes

- Explain the difference between a command-line interface (CLI) and a graphical user interface (GUI).
- Check whether Tkinter is available on a computer and understand what to do when it is not.
- Create and configure a Tkinter root window.
- Use common widgets: Label, Entry, Button, Frame, Radiobutton, Combobox and Listbox.
- Arrange widgets using pack() and grid().
- Use functions as event handlers for button clicks.
- Read user input using .get(), validate it, and provide feedback with message boxes.
- Build a complete Student Registration desktop application and extend it independently.

2. Suggested 2-Hour Workshop Flow

Time	Activity
0-10 min	GUI/Tkinter introduction and setup check
10-25 min	Create first window and understand mainloop()
25-45 min	Labels, Entry fields, Buttons and input retrieval
45-60 min	Layout managers and Frames
60-75 min	Tkinter variables, Radiobuttons and Combobox
75-90 min	Validation and message boxes
90-110 min	Build full Student Registration app
110-120 min	Independent extension challenge and submission check

3. What Is a GUI?

A command-line interface (CLI) is controlled mainly by typed text. A graphical user interface (GUI) lets users interact with visible controls such as windows, buttons, text boxes, menus and lists.

Tkinter is Python's standard interface to the Tk GUI toolkit. It allows Python code to create desktop windows and widgets. The Python logic you already know - variables, functions, conditions, loops, classes and exceptions - is still used; Tkinter adds the visual interface and event handling.

4. How Tkinter Programs Are Different

A normal script often executes from top to bottom and exits. A GUI program creates an interface and then waits for user actions. This is called event-driven programming.

An event can be a button click, keyboard input, a selection from a list, closing a window, or many other user actions. Tkinter's event loop waits for these events and calls the correct function when one occurs.

Key mental model
WINDOW -> WIDGETS -> LAYOUT -> EVENT -> FUNCTION -> INPUT -> VALIDATION -> OUTPUT

5. Setup and Installation Check

Tkinter is included with most standard Python installations on Windows and macOS. On some Linux distributions, Tk support may be packaged separately. The safest first step is to test it.

Step 1 - Open a terminal. Use the VS Code terminal, Command Prompt, PowerShell, Terminal, or another terminal application.

Step 2 - Run the Tkinter test command. Type the command below and press Enter.

```
python -m tkinter
```

If Tkinter is available, a small demonstration window should open. Close it after confirming that it works.

Step 3 - Alternative import test. You can also test that Python can import Tkinter:

```
import tkinter

print("Tkinter is available")
```

If the command fails
First confirm that Python itself works with python --version. If you are using Linux and Tkinter is missing, install the Tk package provided by your Linux distribution, then repeat python -m tkinter. The exact package name differs by distribution.

6. Create the Workshop Project

Step 1 - Create a folder. Create a folder named tkinter_workshop.

```
mkdir tkinter_workshop
cd tkinter_workshop
```

Step 2 - Open it in VS Code. If the code command is available:

```
code .
```

Step 3 - Create a Python file. Inside the folder, create app.py. This will be the main program file.

7. Your First Tkinter Window

Type the following into app.py:

```
import tkinter as tk

window = tk.Tk()

window.mainloop()
```

What each line means

import tkinter as tk imports the Tkinter module and gives it the short name tk. This keeps later code readable, for example tk.Label and tk.Button.

window = tk.Tk() creates the root window. The root window is the main top-level window for the application.

window.mainloop() starts Tkinter's event loop. It keeps the window on the screen and lets Tkinter respond to user actions.

Run the program. Save app.py, then execute:

```
python app.py
```

8. Configure the Window

```
import tkinter as tk

window = tk.Tk()
window.title("Student Registration System")
window.geometry("600x500")

window.mainloop()
```

title() changes the text shown in the operating system title bar. geometry("600x500") sets an initial width of 600 pixels and height of 500 pixels.

9. Understanding Widgets

A widget is a visible GUI component. In Tkinter, widgets are Python objects. You create them, configure them, and then place them inside a parent container.

Widget	Purpose	Used in this workshop
Label	Displays text	Field names and headings
Entry	Single-line input	Name and email
Button	Runs a function when clicked	Register/Clear/Delete
Frame	Groups widgets	Registration form
Radiobutton	Choose one of several values	Gender
ttk.Combobox	Dropdown list	Course
Listbox	Displays multiple items	Registered students

10. Labels and Entry Fields

```
name_label = tk.Label(window, text="Full Name")
name_label.pack()

name_entry = tk.Entry(window)
name_entry.pack()
```

The first argument, window, tells Tkinter which parent should contain the widget. The text option defines the label text. Creating a widget does not automatically place it on the screen; pack() tells Tkinter to place it in the parent.

Reading an Entry value

```
name = name_entry.get()
print(name)
```

.get() returns the current text inside an Entry widget.

11. Buttons and Event Handling

```
def register_student():
    name = name_entry.get()
    print(name)

register_button = tk.Button(
    window,
    text="Register Student",
    command=register_student
)
register_button.pack()
```

> **Important**
> Use command=register_student, not command=register_student(). The first version gives Tkinter the function to call later when the button is clicked. The second version calls the function immediately while building the interface.

12. Practical Task 1 - Name and Age App

Build a small application with two Entry fields and one button. When the button is clicked, retrieve the values and print a sentence.

- Label: Name
- Entry for name
- Label: Age
- Entry for age
- Button: SHOW DETAILS
- Function named show_details()

```
import tkinter as tk

def show_details():
    name = name_entry.get()
    age = age_entry.get()
    print(f"My name is {name} and I am {age} years old.")

window = tk.Tk()
```

```
window.title("Student Information")
window.geometry("400x300")

tk.Label(window, text="Name").pack()
name_entry = tk.Entry(window)
name_entry.pack()

tk.Label(window, text="Age").pack()
age_entry = tk.Entry(window)
age_entry.pack()

tk.Button(window, text="Show Details", command=show_details).pack()

window.mainloop()
```

13. Geometry Managers: pack(), grid() and place()

Tkinter separates creating widgets from deciding where they appear. Geometry managers perform the second job.

pack() - Good for simple stacking or arranging widgets by side.

grid() - Places widgets in rows and columns; excellent for forms.

place() - Uses explicit coordinates. Useful in special cases, but less flexible for beginner form layouts.

```
tk.Label(form_frame, text="Full Name:").grid(row=0, column=0)
name_entry.grid(row=0, column=1)
```

> **Important layout rule**
> Do not use pack() and grid() for different widgets that share the same parent container. You can use pack() in the window and grid() inside a Frame because those widgets have different parents.

14. Frames: Organize the Interface

```
form_frame = tk.Frame(window)
form_frame.pack(pady=10)
```

A Frame is a container. It helps you group related widgets. Our form fields will be children of form_frame, while the title, Register button and results area can be children of the main window.

15. Tkinter Variables and Radio Buttons

Some widgets share a Tkinter variable. Common variable classes are StringVar, IntVar and BooleanVar.

```
gender = tk.StringVar()

tk.Radiobutton(
    form_frame,
    text="Male",
```

```
    variable=gender,
    value="Male"
).grid(row=3, column=1)

tk.Radiobutton(
    form_frame,
    text="Female",
    variable=gender,
    value="Female"
).grid(row=3, column=2)

selected_gender = gender.get()
```

Because both radio buttons use the same variable, selecting one automatically deselects the other. The selected button stores its value into gender.

16. Dropdown Lists with ttk.Combobox

ttk is Tkinter's themed widget set. We will use ttk.Combobox for the Course field.

```
from tkinter import ttk

courses = [
    "Python Development",
    "Embedded Systems",
    "Web Development",
    "Data Analysis"
]

course_combo = ttk.Combobox(
    form_frame,
    values=courses,
    state="readonly",
    width=32
)
course_combo.grid(row=2, column=1)

course = course_combo.get()
```

state="readonly" means the user can select one of the provided choices but cannot type an arbitrary value.

17. Input Validation

Never assume user input is correct. Read the values, clean them, check them, and stop the function if something is invalid.

```
name = name_entry.get().strip()
email = email_entry.get().strip()

if not name:
    print("Name is required")
    return
```

```
if not email:
    print("Email is required")
    return
```

.strip() removes spaces from the beginning and end of a string. return exits the function immediately so invalid data is not processed further.

18. Message Boxes

```
from tkinter import messagebox

messagebox.showerror(
    "Validation Error",
    "Name is required"
)

messagebox.showinfo(
    "Success",
    "Student registered successfully"
)
```

- **showerror() -** Display an error
- **showwarning() -** Display a warning
- **showinfo() -** Display information or success
- **askyesno() -** Ask a Yes/No question and return True or False

19. Practical Task 2 - Add Validation

Improve the Name and Age app so it behaves like a GUI application instead of relying on terminal output.

- Name cannot be empty.
- Age cannot be empty.
- Age must be numeric.
- Invalid input must use messagebox.showerror().
- Valid details must use messagebox.showinfo().

```
def show_details():
    name = name_entry.get().strip()
    age = age_entry.get().strip()

    if not name:
        messagebox.showerror("Error", "Name is required")
        return

    if not age:
        messagebox.showerror("Error", "Age is required")
        return

    if not age.isdigit():
        messagebox.showerror("Error", "Age must be a number")
```

```
        return

    messagebox.showinfo(
        "Student Details",
        f"{name} is {age} years old."
    )
```

20. Final Workshop Project - Student Registration System

Now combine the concepts into one application. The program will collect Full Name, Email, Course and Gender, validate the data, display the student in a Listbox, show a success message, and clear the form.

Complete source code

```
import tkinter as tk
from tkinter import ttk, messagebox


def register_student():
    name = name_entry.get().strip()
    email = email_entry.get().strip()
    course = course_combo.get()
    selected_gender = gender.get()

    if not name:
        messagebox.showerror(
            "Validation Error",
            "Full name is required."
        )
        return

    if not email:
        messagebox.showerror(
            "Validation Error",
            "Email is required."
        )
        return

    if "@" not in email:
        messagebox.showerror(
            "Validation Error",
            "Enter a valid email address."
        )
        return

    if not course:
        messagebox.showerror(
            "Validation Error",
            "Please select a course."
        )
        return

    if not selected_gender:
        messagebox.showerror(
            "Validation Error",
```

```
            "Please select a gender."
        )
        return

    student_text = (
        f"{name} | {email} | "
        f"{course} | {selected_gender}"
    )

    student_list.insert(tk.END, student_text)

    messagebox.showinfo(
        "Success",
        "Student registered successfully."
    )

    clear_form()


def clear_form():
    name_entry.delete(0, tk.END)
    email_entry.delete(0, tk.END)
    course_combo.set("")
    gender.set("")


window = tk.Tk()
window.title("Student Registration System")
window.geometry("750x550")

title_label = tk.Label(
    window,
    text="Student Registration System",
    font=("Arial", 20, "bold")
)
title_label.pack(pady=20)

form_frame = tk.Frame(window)
form_frame.pack(pady=10)

tk.Label(form_frame, text="Full Name:").grid(
    row=0, column=0, padx=10, pady=10, sticky="w"
)
name_entry = tk.Entry(form_frame, width=35)
name_entry.grid(row=0, column=1, padx=10, pady=10)

tk.Label(form_frame, text="Email:").grid(
    row=1, column=0, padx=10, pady=10, sticky="w"
)
email_entry = tk.Entry(form_frame, width=35)
email_entry.grid(row=1, column=1, padx=10, pady=10)

tk.Label(form_frame, text="Course:").grid(
    row=2, column=0, padx=10, pady=10, sticky="w"
)

courses = [
    "Python Development",
    "Embedded Systems",
    "Web Development",
    "Data Analysis"
```

```
]

course_combo = ttk.Combobox(
    form_frame,
    values=courses,
    state="readonly",
    width=32
)
course_combo.grid(row=2, column=1, padx=10, pady=10)

tk.Label(form_frame, text="Gender:").grid(
    row=3, column=0, padx=10, pady=10, sticky="w"
)

gender = tk.StringVar()
gender_frame = tk.Frame(form_frame)
gender_frame.grid(row=3, column=1, sticky="w")

tk.Radiobutton(
    gender_frame,
    text="Male",
    variable=gender,
    value="Male"
).pack(side="left")

tk.Radiobutton(
    gender_frame,
    text="Female",
    variable=gender,
    value="Female"
).pack(side="left")

register_button = tk.Button(
    window,
    text="Register Student",
    command=register_student,
    width=20
)
register_button.pack(pady=15)

tk.Label(
    window,
    text="Registered Students",
    font=("Arial", 14, "bold")
).pack(pady=10)

student_list = tk.Listbox(
    window,
    width=85,
    height=10
)
student_list.pack(padx=20, pady=10)

window.mainloop()
```

21. How the Final Application Works

1. Imports - tkinter supplies the core widgets. ttk supplies the Combobox. messagebox supplies GUI feedback dialogs.

2. register_student() - Reads all form values, validates them, builds one display string, inserts it into the Listbox, shows success feedback, and calls clear_form().

3. clear_form() - Entry.delete(0, tk.END) removes all text. course_combo.set("") resets the dropdown. gender.set("") clears the selected radio button.

4. Form Frame - The form uses grid() because labels and inputs naturally fit into rows and columns.

5. Gender Frame - The gender controls use pack() inside their own child frame. This does not violate the pack/grid rule because the parent is different.

6. Listbox - student_list.insert(tk.END, student_text) appends a new row at the end of the Listbox.

7. mainloop() - The last line starts the event loop. The application then waits for user interactions until the window is closed.

22. Independent Extension Challenge

Choose at least three improvements. The purpose is to prove that you can apply the same pattern independently.

- Add a phone number field.
- Add an age field and validate that age is numeric.
- Add a Clear Form button.
- Add a Delete Selected Student button.
- Display the total number of registered students.
- Prevent duplicate email registration.
- Add more courses.
- Ask for confirmation before deleting a student.
- Improve spacing, fonts or grouping without breaking functionality.

Example: Delete Selected Student

```
def delete_student():
    selected = student_list.curselection()

    if not selected:
        messagebox.showwarning(
            "No Selection",
            "Select a student first."
        )
        return

    confirm = messagebox.askyesno(
        "Confirm Delete",
        "Delete the selected student?"
    )
```

```
    if confirm:
        student_list.delete(selected[0])
```

23. Common Beginner Errors and Fixes

Problem	Likely cause	Fix
Window opens and immediately closes	mainloop() is missing	Make window.mainloop() the final GUI line.
Button function runs before clicking	Used command=my_function()	Use command=my_function without parentheses.
Widget does not appear	Widget created but not placed	Call pack(), grid() or place().
TclError about geometry managers	Mixed pack() and grid() in same parent	Use one geometry manager per parent container.
Entry value is always empty	Read value before user interaction	Call .get() inside the event function.
Radio button has no value	No shared Tkinter variable or value option	Use StringVar and set variable= and value=.
Combobox accepts unexpected text	state not readonly	Use state="readonly" when only listed values are allowed.
python -m tkinter fails	Tk support is unavailable in that Python environment	Check Python installation and install your OS Tk package if necessary.

24. Workshop Submission

Submit a folder with this structure:

```
tkinter_student_registration/
├── app.py
└── README.md
```

- The application launches without errors.
- It contains Labels and Entry fields.
- It contains a Button whose command calls a function.
- It uses a Frame and appropriate layout management.
- It uses a Combobox and Radiobuttons.
- It validates user input and shows message boxes.
- It displays registered students in a Listbox.
- It clears/resets the form after successful registration.
- It includes at least two independent improvements.

Final pattern to remember
Create the window -> create widgets -> arrange widgets -> start mainloop -> wait for an event -> call a function -> get input -> validate -> update the interface.

25. Quick Knowledge Check

1. What does tk.Tk() create?
2. Why is mainloop() necessary?
3. What is a widget?
4. What does Entry.get() return?
5. Why should command=register_student not contain parentheses?
6. When would you choose grid() instead of pack()?
7. What problem does a Frame solve?
8. What is StringVar used for?
9. Why validate user input?
10. What is the role of messagebox.showerror()?

26. Reference Notes

Python documentation describes tkinter as Python's interface to the Tcl/Tk GUI toolkit, and recommends python -m tkinter as a quick installation check. It also identifies widgets, widget hierarchy, geometry management and the event loop as core Tkinter concepts. ttk provides the modern themed widget set used here for Combobox.

Official reference: Python Standard Library documentation - tkinter and tkinter.ttk.